

HDOC 28

NAME = ARAYANA KHARE

BATCH = 15

Question 1

Problem

Write a C program to:

- Read rows and columns and ensure both are positive.
- Declare a **Variable Length 2D Array (VLA)**.
- Read elements **column by column**.
- Display the elements in **row-major order using a single loop and linear index**.

Algorithm

1. Start.
2. Declare `rows`, `cols`, `i`, `j`, and `index`.
3. Input `rows`.
4. If `rows <= 0`, display an error and repeat until a positive value is entered.
5. Input `cols`.
6. If `cols <= 0`, display an error and repeat until a positive value is entered.
7. Declare the VLA as `int a[rows][cols]`.
8. Set `j = 0`.
9. Repeat for every column:
 - Set `i = 0`.
 - Repeat for every row.
 - Read `a[i][j]`.
10. To display in row-major order, run one loop with `index` from 0 to `rows * cols - 1`.
11. Access each element as:
`a[index / cols][index % cols]`.
12. Print a newline whenever `(index + 1) % cols == 0`.
13. Stop.

Test Case Table

TestCase

Input Expected

Output Actual

Output

Result

1 rows=2, cols=3, elements=1 2

3 4 5 6 1 3 5 / 2 4 6 1 3 5 / 2 4 6 Pass

2 rows=1, cols=1, element=10 10 10 Pass

3 rows=0, then 2; cols=2 Error for rows, then
accepts 2

Error displayed, then

accepts 2 Pass

4 rows=2; cols=-3, then 2 Error for columns, then
accepts 2

Error displayed, then

accepts 2 Pass

Note: In Test Case 1, input 1 2 3 4 5 6 is entered **column by column**, so the array becomes:

1 3 5

2 4 6

C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int rows, cols, i, j, index;
```

```
/* Read and validate number of rows */
```

```
do
```

```
{
```

```
printf("Enter number of rows: ");
```

```
scanf("%d", &rows);
```

```
if (rows <= 0)
```

```
{
```

```
printf("Invalid input! Rows must be positive.
```

```
\n");
```

```
}
```

```
} while (rows <= 0);
```

```
/* Read and validate number of columns */
```

```
do
```

```
{
```

```
printf("Enter number of columns: ");
```

```
scanf("%d", &cols);
```

```

if (cols <= 0)
{
printf("Invalid input! Columns must be positive.
\n");
}
} while (cols <= 0);
/* Variable Length 2D Array */
int a[rows][cols];
/* Input elements column by column */
printf("\nEnter the array elements column by column:\n");
for (j = 0; j < cols; j++)
{
for (i = 0; i < rows; i++)
{
printf("a[%d][%d] = ", i, j);
scanf("%d", &a[i][j]);
}
}
/* Display in row-major order using a single loop */
printf("\nArray in row-major order:\n");
for (index = 0; index < rows * cols; index++)
{
printf("%d ", a[index / cols][index % cols]);
if ((index + 1) % cols == 0)
{
printf("\n");
}
}
return 0;
}

```

Execution

```
Enter number of rows: 2
Enter number of columns: 3

Enter the array elements column by column:
a[0][0] = 0 1
a[1][0] = a[0][1] = 12
a[1][1] = 22
a[0][2] = 34
a[1][2] = 34

Array in row-major order:
0 12 34
1 22 34
```

Question 2

Problem

Write a C program that:

1. 2. 3. Reads elements into a **variable-length 2D array**.
- Copies all elements into a **1D array in row-major order**.
- Displays both the 2D and 1D arrays.

Algorithm

1. Start.
2. Read `rows` and `cols`.
3. Validate that both are positive.
4. Declare `int a[rows][cols]`.
5. Declare `int b[rows * cols]`.
6. Read elements of the 2D array using nested loops.
7. Set `index = 0`.
8. Traverse the 2D array row by row.
9. Copy each element using `b[index] = a[i][j]`.
10. Increment `index`.
11. Display the 2D array.
12. Display the 1D array.
13. Stop.

Test Case Table

Test

Case Input Expected

1D Array Actual

Output

Result

1 2×3: 1 2 3 4 5

6 1 2 3 4 5 6 1 2 3 4 5

6

Pass

2 2×2: 10 20 30

40 10 20 30 40 10 20 30

40

Pass

3 1×3: 5 10 15 5 10 15 5 10 15 Pass

4 rows=-2, then 2 Error, then accepts valid

value Error displayed Pass

5 cols=0, then 2 Error, then accepts valid

value Error displayed Pass

C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int rows, cols, i, j, index;
```

```
/* Input and validate rows */
```

```
do
```

```
{
```

```
printf("Enter number of rows: ");
```

```
scanf("%d", &rows);
```

```
if (rows <= 0)
```

```
{
```

```
printf("Invalid input! Rows must be positive.
```

```
\n");
```

```
}
```

```
} while (rows <= 0);
```

```
/* Input and validate columns */
```

```
do
```

```
{
```

```
printf("Enter number of columns: ");
```

```
scanf("%d", &cols);
```

```
if (cols <= 0)
```

```
{
```

```

printf("Invalid input! Columns must be positive.
\n");
}
} while (cols <= 0);
/* Variable Length Arrays */
int a[rows][cols];
int b[rows * cols];
/* Read 2D array */
printf("\nEnter the elements of the 2D array:\n");
for (i = 0; i < rows; i++)
{
for (j = 0; j < cols; j++)
{
printf("a[%d][%d] = ", i, j);
scanf("%d", &a[i][j]);
}
}
/* Copy 2D array into 1D array in row-major order */
index = 0;
for (i = 0; i < rows; i++)
{
for (j = 0; j < cols; j++)
{
b[index] = a[i][j];
index++;
}
}
/* Display 2D array */
printf("\n2D Array:\n");
for (i = 0; i < rows; i++)
{
for (j = 0; j < cols; j++)
{
printf("%d ", a[i][j]);
}
printf("\n");
}
}

```

```
/* Display 1D array */
printf("\n1D Array in row-major order:\n");
for (index = 0; index < rows * cols; index++)
{
    printf("%d ", b[index]);
}
printf("\n");
return 0;
}
```

Execution

```
Enter number of rows: 2
Enter number of columns: 3

Enter the elements of the 2D array:
a[0][0] = 10
a[0][1] = 20
a[0][2] = 30
a[1][0] = 40
a[1][1] = 50
a[1][2] = 60

2D Array:
10 20 30
40 50 60

1D Array in row-major order:
10 20 30 40 50 60
```